

MTE 325 Lab notes

By: Khalil (21074013), Yaseen Mohamed (21069303), Uzayr, Phong

Lab Session 1:

- After reading the schematic, Pin PA9 was the unused pin that was configured as the GPIO input for the switch. (All members discussed to achieve a conclusion, then wrote the code.)
- We were unable to read the value of the pin at first, but after debugging, we realized it was because we forgot the “volatile” keyword. (All members helped with debugging and had code running on their devices.)
- PA10 was another unused pin that was configured (as output) for the LED. (All the members discussed.)
- The LED was toggled on, wait some time, toggle off, wait some time. (All members, with different code variations on each device tested their code.)
- Motor control was not working at first, but after consulting the TAs, they helped us realize that the wrong motor number was being used in code (we were trying with M0, but once we used M1, then it worked properly). (All members attempted code; Khalil was able to get working after all members debugged.)

Acknowledgements: No external help, only TA assistance (TA assistance for motor control debugging)

Lab Session 2:

Exercise 5.1

1st step: (Phong and Yaseen built and debugged circuit, with help from all team members), (Khalil and Uzayr set up the oscilloscope, and yaseen and Phong connected to the circuit.)

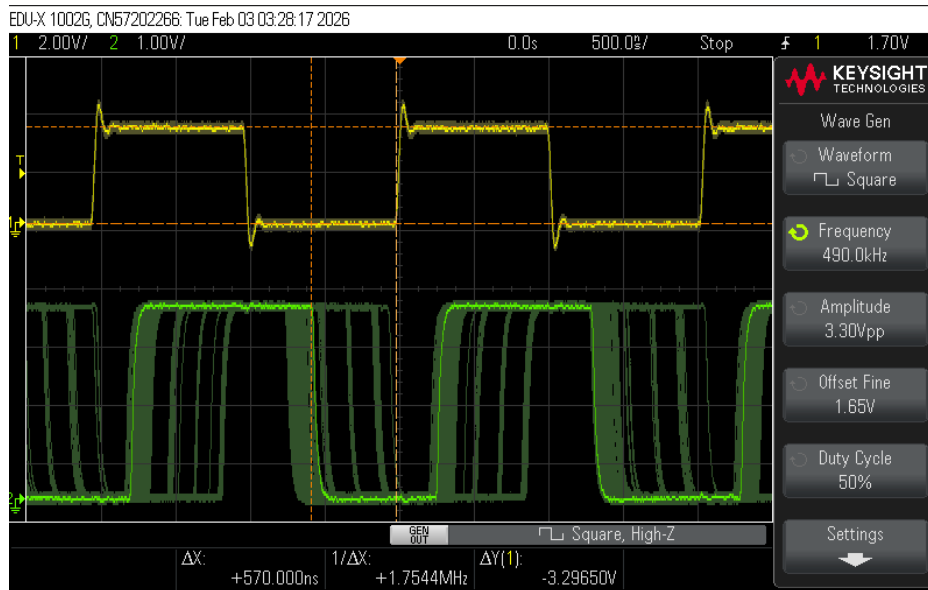
- Construct the LED circuit connected to the STM32 board using pin PA9 on the board.
- Then configured the signal generator to create a square waveform, 100hz frequency, amplitude of 3.30Vpp, and offset of 1.65V.
- Connect the positive probe of oscilloscope (signal generator) to D8 pin of the STM32 and negative probe to the ground line.

2nd step: (Tight poll code written by Yaseen and Uzayr and debugged by rest of the team to fix errors and make it work, and Phong and Khalil worked on oscilloscope max frequency and variation in time response analysis, with help from all members)

- We wrote code to Tight poll using a while loop, for the value at PA8
- We check if the pin is “reset”, which is low, then as soon as it is low, its “sets” or high to PA9, then repeat the same but in reverse

Problems we ran into were accidentally wiring the wrong ground and messing up the polarity of the LED connection.

In the code, the input pin was wrongfully flipped with the output when connecting to the circuit; the input and output pins were initialized to be flipped (input as output and output as input). So, the connection was flipped, and implementation was also flipped. To fix it, we read each pin separately, then reconnected the pins according to the code and compared pin initialization with the polling, reading, and resetting code.



We found the maximum frequency through increasing the frequency until the LED square wave output does not turn off before the next rising edge of the signal generator square wave.

The reliable response was determined when the LED goes low before the next rising edge of the signal generator.

Variation in response time with persistence time of 200ms: **570ns**

The reason for this variation is due to the frequency being too high for the tight polling implementation, and because of the delay added in our code, if the delay is longer, then the variation should be less.

490 khz is the maximum frequency with which we are reliably able to respond to using polling.

The code at this step:

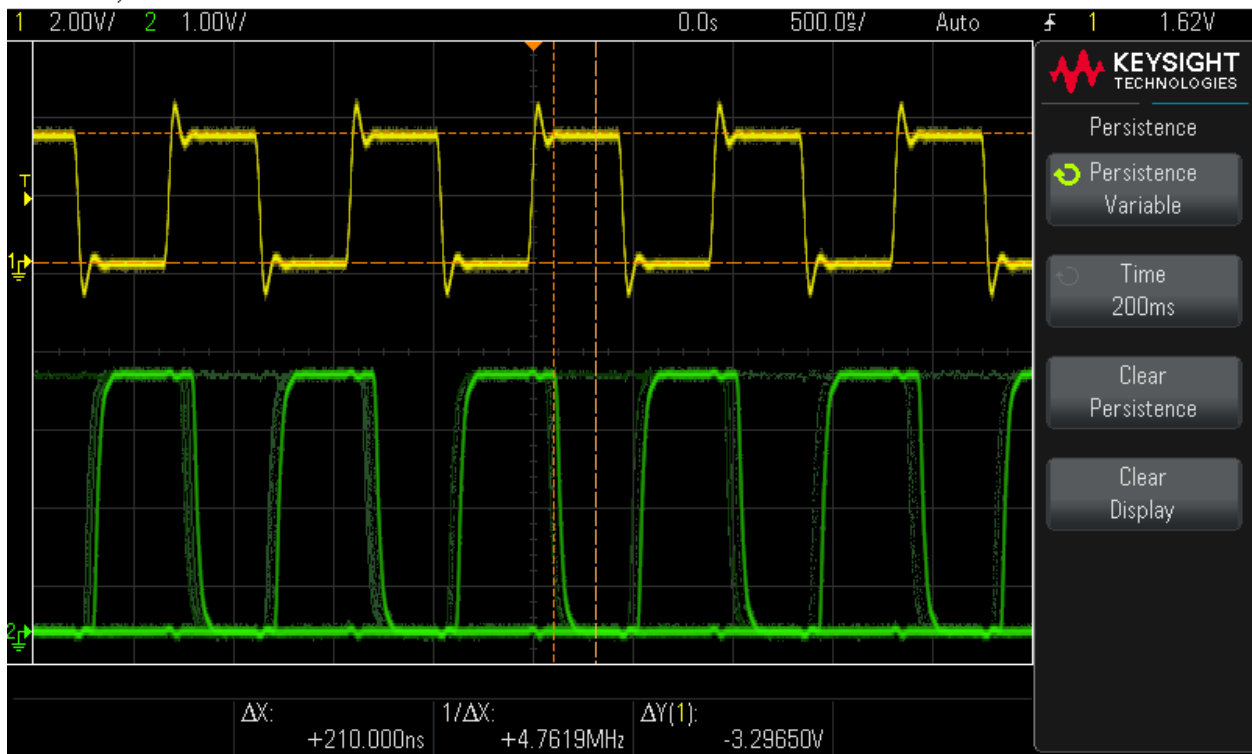
```
while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8) == GPIO_PIN_RESET) { }  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET); while  
(HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8) == GPIO_PIN_SET) { }  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET);
```

Exercise 5.2

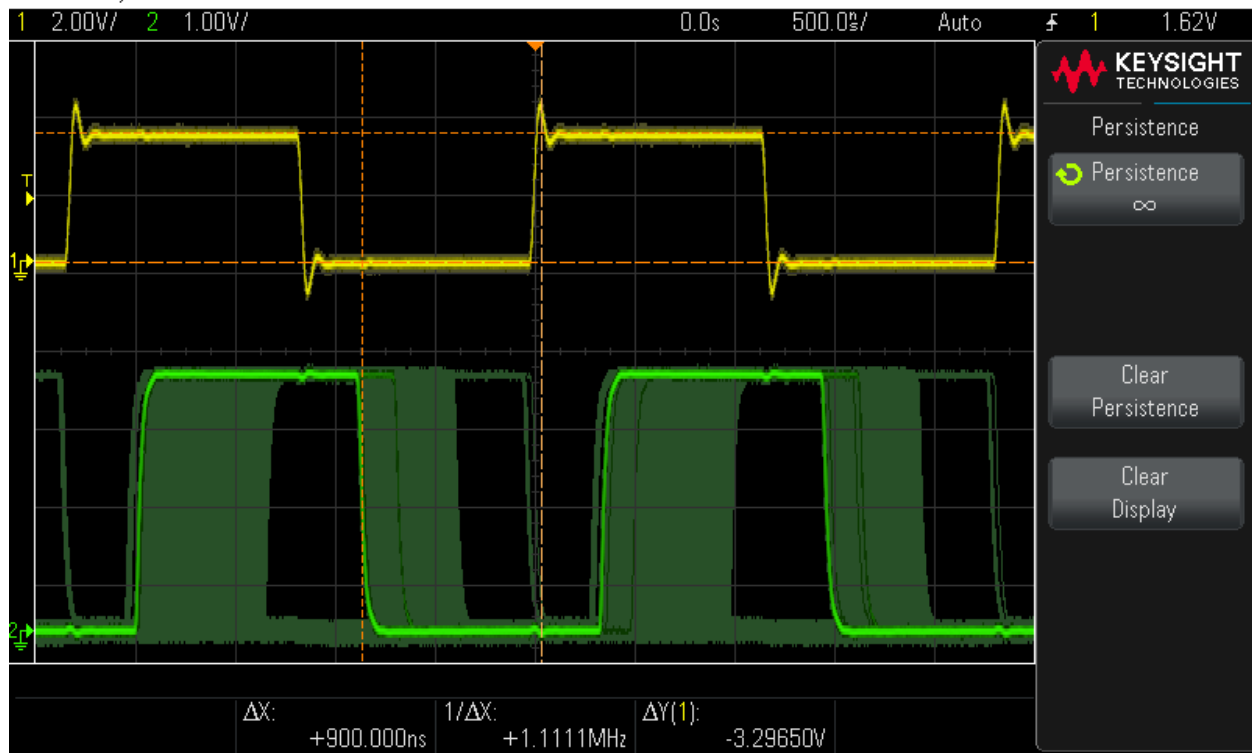
(Interrupt code written by Uzayr and debugged yaseen with help of the rest of the team to fix errors and make it work, and Phong and Khalil worked on oscilloscope max frequency and variation in time response analysis, with help from all members.)

In our interrupt we need to match what we did for the polling case (rise on rise and fall on falling) (PA10 was used for the interrupt GPIO input, instead of PA8); in our interrupt it was the opposite of the polling, so when we have a rising edge interrupt, we set the LED to turn on and turn it off before the next rising edge.

EDU-X 10026, CN57202266: Tue Feb 03 04:04:35 2026



EDU-X 1002G, CN57202266, Tue Feb 03 04:09:20 2026



590 kHz is the maximum frequency with which we are reliably able to respond to using interrupts.

Variation in response time with persistence time of 200ms: **900ns**

Acknowledgements: No external help, only TA assistance (TA assistance with debugging errors in initial circuit connections and both codes, as well as advising that interrupts can be done on the rising edge, then tight polling on the falling edge, or interrupts on both. The TA assisted by explaining the use of the oscilloscope to get max frequency and response time variation.)

Lab Session 3:

Initial approach is to detect the falling edge of the limit switch, trigger an interrupt and stop the motor.

Choosing the falling edge since the switch is normally closed, the switch will be connected to the input 3.3V from the Arduino connected to the motor, when the switch opens a falling edge is triggered and the interrupt will happen then.

(yasin and phong did the wiring)

We set the power supply 12V and limit current will be 1A.

(Khalil, phong and uzayur looked at schematics to find empty pins)

Determine the pin that free to use. First step: check on the code if that pin is already used for other purposes. After that, we check on the motor shield to see if that pin have been used or not. If not, that pin is good to use. Observation: The Pin with B is more available than Pin with A and C. Eventually, we find: PA8, PA9, PA10, PB8, PB9, PB10....are available.

First step is to know that we will only use the pins on the motor shield, then cross referencing from the motor shield schematic, we identified that PB6, PA9, PA10, PA11 are free on the motor shield and can be used. These are the pins we will be using.

In the code we created a new function that handles lines 8 and 9 and 6.

(everyone helped write the code)

@brief This function handles EXTI Line[9:5] interrupts.

*/

```
void EXTI9_5_IRQHandler(void)
```

```
{
```

```
    if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_9) != RESET)
```

```
    {
```

```
        HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_9);
```

```

}
if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_8) != RESET)
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_8);
}
if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_6) != RESET)
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_6);
}
}

```

(yasin and uzayr tested the unmounted motors with the limit switches)

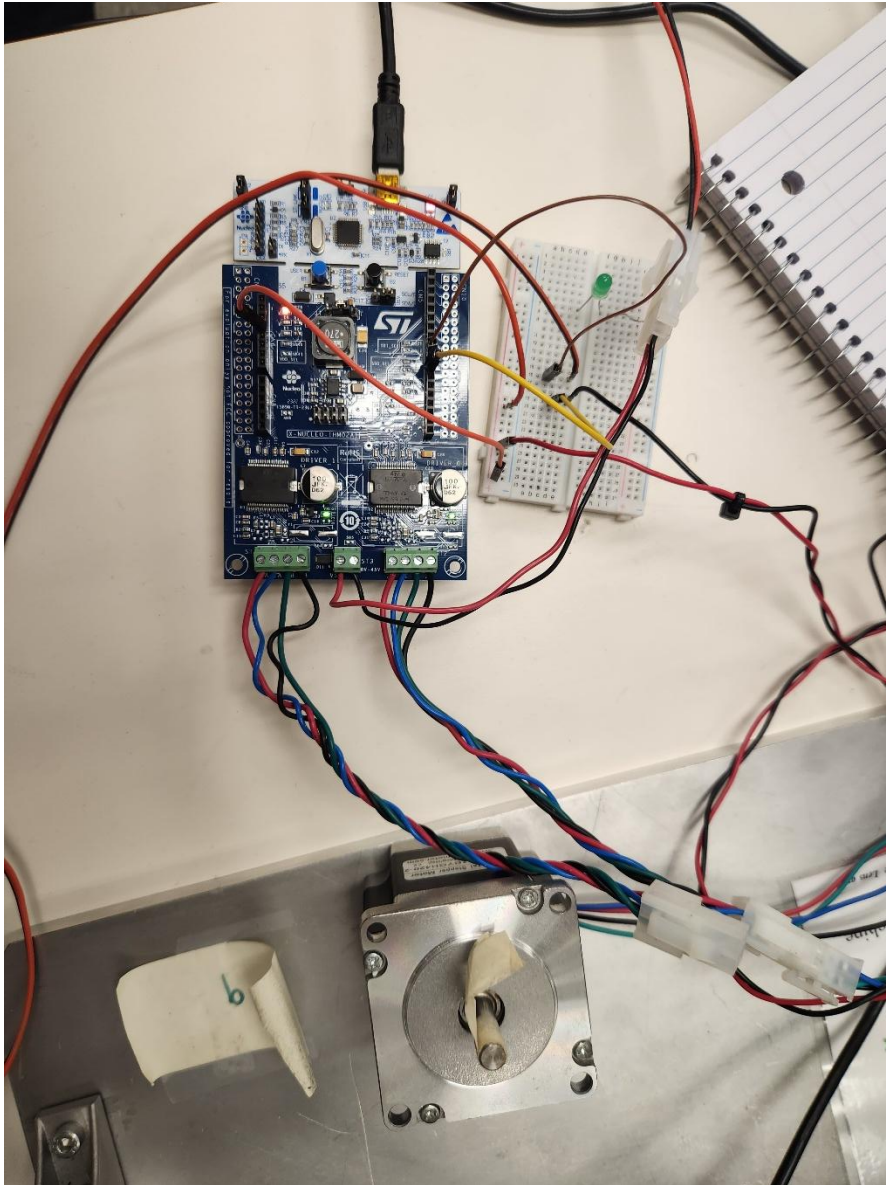
Then set priority of all pins to subpriority 0, which means highest priority.

The first step, we connect the unmouted motor to test if we can roate it and when hit limit switch if it turn on or not

(Khalil and uzayr branstorms brainstormed testcases and how to implement them)

Edge cases to consider:

- First case is if 2 limit switches are pressed at the same time (already accounted for)
- If the program flashes and starts off with a limit switch being pressed



Adding the interrupt logic.

Declaring pin A8, A9, A10, A11 as falling edge triggered interrupts in the interface.c file

```
{  
#ifdef NUCLEO_USE_USER_BUTTON  
    /* Configures Button GPIO and EXTI Line */  
    BSP_PB_Init(BUTTON_KEY, BUTTON_MODE_EXTI);  
#endif
```



```
#ifdef NUCLEO_USE_USER_LED
```

```
/* Configures LED GPIO */
```

```
BSP_LED_Init(LED2);
```

```
#endif
```

```
__HAL_RCC_GPIOA_CLK_ENABLE();
```

```
__HAL_RCC_GPIOB_CLK_ENABLE();
```

```
/* Declare a struct, which is then to be specified to configure an GPIO Pin */
```

```
GPIO_InitTypeDef GPIO_InitStructure = {0};
```

```
GPIO_InitStructure.Pin = GPIO_PIN_8; // Change X to your pin
```

```
GPIO_InitStructure.Mode = GPIO_MODE_IT_FALLING;
```

```
GPIO_InitStructure.Pull = GPIO_PULLDOWN; // Use pulldown when the switch is connected  
to 3.3V
```

```
GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_LOW;
```

```
HAL_GPIO_Init(GPIOA, &GPIO_InitStructure); // Sends the configuration to specified Port and  
Pin settings
```

```
HAL_NVIC_SetPriority(EXTI9_5_IRQn, 2, 0);
```

```
HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);
```

```
GPIO_InitTypeDef GPIO_InitStructure_1 = {0};
```

```
GPIO_InitStructure_1.Pin = GPIO_PIN_9; // Change X to your pin
```

```
GPIO_InitStructure_1.Mode = GPIO_MODE_IT_FALLING;
```

```
GPIO_InitStructure_1.Pull = GPIO_PULLDOWN; // Use pulldown when the switch is  
connected to 3.3V
```

```
GPIO_InitStructure_1.Speed = GPIO_SPEED_FREQ_LOW;
```

```
HAL_GPIO_Init(GPIOA, &GPIO_InitStructure_1); // Sends the configuration to specified Port  
and Pin settings
```

```
HAL_NVIC_SetPriority(EXTI9_5_IRQn, 2, 0);
```

```
HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);
```

```
GPIO_InitTypeDef GPIO_InitStruct_2 = {0};
```

```
GPIO_InitStruct_2.Pin = GPIO_PIN_6; //Change X to your pin
```

```
GPIO_InitStruct_2.Mode = GPIO_MODE_IT_FALLING; //Push-Pull output, meaning the pin  
supplies 3.3V and 0V
```

```
GPIO_InitStruct_2.Pull = GPIO_PULLDOWN; //No internal pullup/down for output
```

```
GPIO_InitStruct_2.Speed = GPIO_SPEED_FREQ_LOW;
```

```
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct_2);
```

```
HAL_NVIC_SetPriority(EXTI15_10_IRQn, 2, 0);
```

```
HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
```

```
GPIO_InitTypeDef GPIO_InitStruct_3 = {0};
```

```
GPIO_InitStruct_3.Pin = GPIO_PIN_10; //Change X to your pin
```

```
GPIO_InitStruct_3.Mode = GPIO_MODE_IT_FALLING; //Push-Pull output, meaning the pin  
supplies 3.3V and 0V
```

```
GPIO_InitStruct_3.Pull = GPIO_PULLDOWN; //No internal pullup/down for output
```

```
GPIO_InitStruct_3.Speed = GPIO_SPEED_FREQ_LOW;
```

```
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct_3);
```

```
HAL_NVIC_SetPriority(EXTI15_10_IRQn, 2, 0);
```

```
HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
```

```
}
```

We can connection the second motor and using PB6 and PB10 for interputation

The next step is checking the edge cases in main, on startup. This is done to avoid any edge cases being missed and causing unexpected behavior on running. Edge cases being accounted for are:

- Both limit switches on one of the Axes are pressed.
 - That indicates possible faulty hardware, connections, or software issue that caused this detection.
 - The solution was to fully stop both motors and set a flag to indicate the fault and avoid any motor being driven, and print an error serially, allowing the user to debug and fix the fault, and avoid any equipment damage.
- One of the limit switches is pressed on startup.
 - That indicates the motor is at one of the two sides, therefore, it starts moving in the opposite direction (assuming the position of switches and required direction after triggering each is known).

Note: if any fail cases is triggered (both switches triggered on the same axis)

Code:

```
{  
/*read all limit switch values*/  
  
GPIO_PinState SW1_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_10);  
GPIO_PinState SW2_value = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_6);  
GPIO_PinState SW3_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9);  
GPIO_PinState SW4_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8);  
  
bool motor_0_set = false;  
bool motor_1_set = false;  
  
//ensure fault is reset and not detected at the start of the program  
fault_detected = false;  
  
// horizontal axis
```

```

if (SW3_value == GPIO_PIN_RESET && SW4_value == GPIO_PIN_RESET) //motor 0 both
switches triggered

{
    L6470_Debounce_Delay(debounce_delay_ms);

    SW3_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9);

    SW4_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8);

    if (SW3_value == GPIO_PIN_RESET && SW4_value == GPIO_PIN_RESET) //motor 0 both
switches still triggered after debounce delay

    {

        L6470_HardStop(0);

        L6470_HardStop(1);

        fault_detected = true;

        USART_Transmit(&huart2, "Fault detected: both motor 0 switches triggered fix issue and
reset\r\n");

        motor_0_set = true;

        motor_1_set = true;

    }

}

if (SW3_value == GPIO_PIN_RESET && motor_0_set == false && fault_detected == false)
//motor 0 right switch triggered PA9

{

    L6470_Debounce_Delay(debounce_delay_ms);

    SW3_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9);

    if (SW3_value == GPIO_PIN_RESET) //motor 0 right switch still triggered after debounce
delay

    {

        L6470_Run(0, L6470_DIR_FWD_ID, 3000);

```

```

    motor_0_set = true;

}

}

if (SW4_value == GPIO_PIN_RESET && motor_0_set == false && fault_detected == false)
//motor 0 left switch triggered PA8

{

    L6470_Debounce_Delay(debounce_delay_ms);

    SW4_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8);

    if (SW4_value == GPIO_PIN_RESET) //motor 0 left switch still triggered after debounce
delay
    {

        L6470_Run(0, L6470_DIR_REV_ID, 3000);

        motor_0_set = true;

    }

}

if (motor_0_set == false && fault_detected == false) //motor 0 no switch triggered

{

    L6470_Run(0, L6470_DIR_FWD_ID, 3000);

    motor_0_set = true;

}


// vertical axis

if (SW1_value == GPIO_PIN_RESET && SW2_value == GPIO_PIN_RESET && fault_detected
== false) //motor 1 both switches triggered

{

    L6470_Debounce_Delay(debounce_delay_ms);

    SW1_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_10);

```

```

SW2_value = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_6);

if (SW1_value == GPIO_PIN_RESET && SW2_value == GPIO_PIN_RESET) //motor 1 both
switches still triggered after debounce delay

{
    L6470_HardStop(1);

    L6470_HardStop(0);

    USART_Transmit(&huart2, "Fault detected: both motor 1 switches triggered fix issue and
reset\r\n");

    fault_detected = true;

    motor_0_set = true;

    motor_1_set = true;

}

}

if (SW1_value == GPIO_PIN_RESET && motor_1_set == false && fault_detected == false)
//motor 1 top switch triggered PA10

{
    L6470_Debounce_Delay(debounce_delay_ms);

    SW1_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_10);

    if (SW1_value == GPIO_PIN_RESET) //motor 1 top switch still triggered after debounce
delay

    {
        L6470_Run(1, L6470_DIR_FWD_ID, 30000);

        motor_1_set = true;

    }

}

if (SW2_value == GPIO_PIN_RESET && motor_1_set == false && fault_detected == false)
//motor 1 bottom switch triggered PA6

```

```

{
    L6470_Debounce_Delay(debounce_delay_ms);
    SW2_value = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_6);
    if (SW2_value == GPIO_PIN_RESET) //motor 1 bottom switch still triggered after
    debounce delay
    {
        L6470_Run(1, L6470_DIR_REV_ID, 30000);
        motor_1_set = true;
    }
}
if (motor_1_set == false && fault_detected == false) //motor 1 no switch triggered
{
    L6470_Run(1, L6470_DIR_FWD_ID, 30000);
    motor_1_set = true;
}
}

```

Next, the interrupt handler was setup for each pin to set the direction of the motor opposite to that switch (this assumes the position vs direction of motor is known and correct for each motor). This was done using the code below:

```

{
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    switch (GPIO_Pin)
    {
        case GPIO_PIN_9: // horizontal axis right switch
            L6470_Debounce_Delay(debounce_delay_ms);

```

```

    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9) == GPIO_PIN_RESET && fault_detected ==
false)
    {
        L6470_Run(0, L6470_DIR_FWD_ID, 3000);
    }

    break;
case GPIO_PIN_8: // horizontal axis left switch

    L6470_Debounce_Delay(debounce_delay_ms);

    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8) == GPIO_PIN_RESET && fault_detected ==
false)
    {
        L6470_Run(0, L6470_DIR_REV_ID, 3000);
    }

    break;
case GPIO_PIN_10: // vertical axis top switch

    L6470_Debounce_Delay(debounce_delay_ms);

    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_10) == GPIO_PIN_RESET && fault_detected ==
false)
    {
        L6470_Run(1, L6470_DIR_FWD_ID, 30000);
    }

    break;
case GPIO_PIN_6: // vertical axis bottom switch

    L6470_Debounce_Delay(debounce_delay_ms);

    if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_6) == GPIO_PIN_RESET && fault_detected ==
false)
    {

```



```

    L6470_Run(1, L6470_DIR_REV_ID, 30000);
}
break;
case GPIO_PIN_13:
    BSP_EmergencyStop();
    break;
case L6470_nBUSY_SYNC_GPIO_PIN:
    BSP_L6470_BusySynchEventManager();
    break;
case L6470_nFLAG_GPIO_PIN:
    BSP_L6470_FlagEventManager();
    break;
}
}
}

```

Where the handler call was initialized using this code:

```

{
void EXTI15_10_IRQHandler(void)
{
    if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_10) != RESET)
    {
        HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_10);
    }

    if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_13) != RESET)
    {

```

```

    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_13);
}
}

/**
 * @brief This function handles EXTI Line[9:5] interrupts.
 */
void EXTI9_5_IRQHandler(void)
{
    if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_9) != RESET)
    {
        HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_9);
    }
    if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_8) != RESET)
    {
        HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_8);
    }
    if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_6) != RESET)
    {
        HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_6);
    }
}
}

```

This code uses the L6470_Run function to control the direction and speed of the motors. This function has an internal check for the fault_detected Boolean that ensures the motor does not run if it is in this mode, to avoid equipment damage.

Lab 4 session:

Part 1:

From lab 3 code, add a debounce to the limit switches (15ms delay then check switch again), to ensure the switch is actually triggered, and cleanup noise in the clicks. The process is:

- Delay 15ms
- Read button values again
- Check if still triggered, and apply conditions if that is the case.

Note: this is applied to edge cases, as well as interrupts.

Example interrupt debounced code:

```
{
case GPIO_PIN_10:
    HAL_Delay(SW_debounce_delay_ms); //delay to ensure the switch value is stable
    GPIO_PinState switch_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_10);
    if (switch_value == GPIO_PIN_RESET)
    {
        L6470_Run(1, L6470_DIR_REV_ID, 3000);
    }
    break;
}
```

After testing and implementing this code, we found that the HAL_Delay function caused the code to break as it itself is also interrupt driven and was causing interference with our switch interrupts. For the demo we switched to a pure stopping the motor case instead of reversing the direction.

We implemented cases where we check the value of any switch, if it is pressed, we stop the corresponding motor, the 2 switches on x axis correspond to one motor and the 2 on the y axis correspond to the other motor. We also are not implementing any debouncing for this approach as in the debouncing we would check if any given switch were pressed, then wait, and check again just to stop. Without the debouncing we just receive a signal and stop the

motor, even with debouncing we would just check and stop again.

We then worked on implementing the logic to reverse the motor direction after a switch click. The main logic follows that we know the location of each switch and the direction desired when it is pressed. Switches 1 to 4 are hardcoded locations, we setup the interrupt cases for each switch to move the corresponding motor in the opposite direction from that switch rather than toggling. This mostly eliminates the need for debouncing as at any slight switch trigger the motor is set to move in the away direction from the button (does not matter if it was one press or multiple), now when running the code while it is at any corner this case is handled as well, since both motors would just go in the opposite corner at any switch trigger. (the code and cases matches the final implementation explained in lab 3)

Then there was a possibility of any noise on the lines being seen as a momentary button press and switching the direction of the motor unintentionally, which is a very unlikely case, but it is better to be handled now. To fix this, a debounce was added, with a custom L6470_Debounce_Delay function added to the stepper motors' functions, which uses a for loop to loop for 20000 times for each millisecond, approximating the delay and not interfering with the interrupts, to avoid all delay interrupt issues:

```
void L6470_Debounce_Delay(uint32_t delay_ms){  
    uint32_t t = delay_ms*20000;  
    for (int i = 0; i < t; i++) {}  
};
```

This was then implemented in the debounce for the buttons as follows:

```
case GPIO_PIN_10: // vertical axis top switch  
    L6470_Debounce_Delay(debounce_delay_ms);  
    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_10) == GPIO_PIN_RESET && fault_detected ==  
false)  
    {  
        L6470_Run(1, L6470_DIR_FWD_ID, 30000);  
    }  
    break;
```

This lead to buttons being debounced, and will only be triggered when actually pressed, not with any outside noise.

To add: add a check when an interrupt is fired to check if the other switch in that axis is also pressed, as in such a case, there is an issue with the system and it needs to completely stop all motors for safety of equipment.

Part 2:

We figured out how to connect the Potentiometer to board as the picture showing below.



We found the ADC pin

28	PA0	A0	1
30	PA1	A1	2
32	PA4	A2	3
34	PB0	A3	4
36	PC1	A4	5
38	PC0	A5	6

CN8

We use PA0

Steps:

1. Activate Internal ADC

- What is needed to turn it on?:

To be continued next lab.